React Animations com Framer Motion

Exemplo prático: Pokédex Motion

Programação Full Stack · Prof. Edécio Fernando Iepsen

UniSenac Pelotas · Trabalho #2 – Pacotes Extras

Integrantes: Mateus Zanatta e Maiara Torchelsen



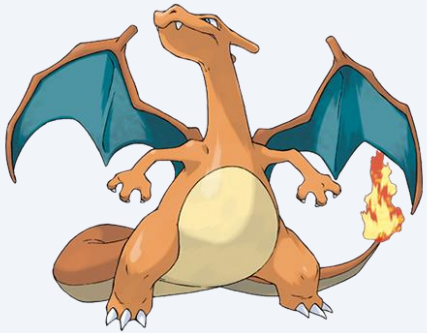
O que é o Framer Motion?

Biblioteca de animações para React: em vez de escrever CSS com @keyframes, você descreve como o elemento deve ficar e ela cuida da animação.

Declarativo
Você diz o estado inicial e o final; o caminho é calculado.

Física de verdade
Molas (spring), inércia e arrasto deixam o movimento natural.

Feito para React
Basta trocar por <motion.div> e passar props.



npm: framer-motion
Hoje o projeto também se chama "Motion" (pacote motion). A API é a mesma.

Instalação e 1º exemplo

1. Instalar no projeto React (front)

```
npm install framer-motion
```

2. Importar e usar o componente motion

initial como o elemento começa

animate como ele deve terminar

transition como ir de um ao outro

```
import { motion } from "framer-motion"

export function Caixa() {
  return (
    <motion.div
      initial={{ opacity: 0, y: 50 }}
      animate={{ opacity: 1, y: 0 }}
      transition={{ duration: 0.5 }}
    />
  )
}

// a div aparece subindo e
// ficando visível em 0,5s
```

Os conceitos que usamos



Gestos

whileHover, whileTap e drag reagem ao mouse e ao toque



Variants

Estados com nome que se propagam do pai para os filhos



AnimatePresence

Permite animar a SAÍDA de um elemento (exit)



Layout e layoutId

Anima mudanças de posição e tamanho automaticamente



Keyframes e loops

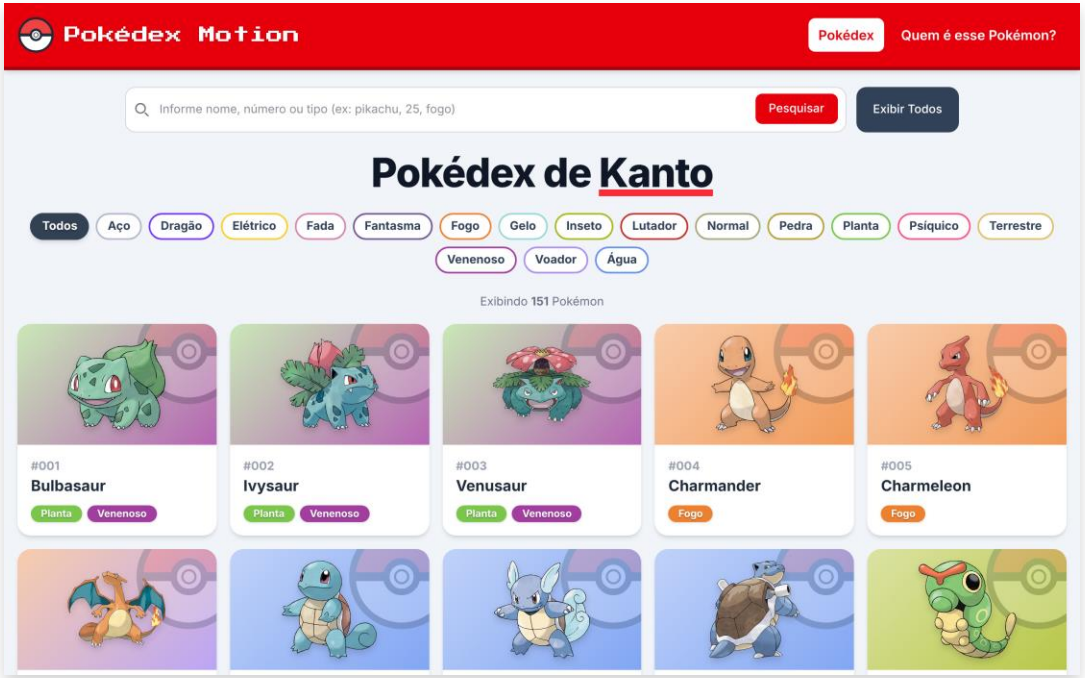
Arrays de valores e repeat: Infinity



Hooks

useScroll, useSpring, useTransform e useAnimate

O projeto: Pokédex Motion



PostgreSQL
151 Pokémon + 17 tipos (seed)

API: Express + Prisma
/pokemons · /tipos · /sorteio

Front: React + Vite
Tailwind + Framer Motion

Mesma estrutura dos exemplos da aula: pastas back/ e front/

Estrutura de pastas

```
framer-motion/
├─ back/
│  ├─ prisma/
│  │  ├─ schema.prisma   // tabelas
│  │  ├─ dados.ts        // 151 Pokémon
│  │  └─ seed.ts         // popula o banco
│  ├─ lib/prisma.ts      // conexão
│  └─ src/
│     ├─ server.ts       // Express
│     └─ routes/         // pokemons, tipos
└─ front/src/
   ├─ main.tsx           // rotas
   ├─ Layout.tsx         // menu + transição
   ├─ App.tsx            // Pokédex
   ├─ Detalhes.tsx       // página do Pokémon
   ├─ Quiz.tsx           // Quem é esse Pokémon?
   ├─ components/        // animações
   └─ utils/             // tipos TypeScript
```



back/

Express + Prisma + Zod, igual às aulas. O seed cadastra os 151 Pokémon.



front/

React + Vite + Tailwind + React Router + Hook Form + Sonner.



Pacote novo

Só o framer-motion, instalado no front-end.

Como o projeto funciona

1. **Banco de dados**
   O seed grava 17 tipos e 151 Pokémon no PostgreSQL.

2. **API (back-end)**
   O Express recebe GET /pokemons e o Prisma busca no banco.

3. **Busca no front**
   O App.tsx faz o fetch no useEffect e guarda no useState.

4. **Tela**
   Cada Pokémon vira um <CardPokemon>, que é um motion.div.

5. **Animação**
   O Framer Motion anima a entrada, o filtro, o hover e a saída.

```
// App.tsx (resumido)
const [pokemons, setPokemons] =
  useState<PokemonType[]>([])

useEffect(() => {
  buscaDados()  // GET /pokemons
}, [])

<AnimatePresence mode="popLayout">
  {filtrados.map(p => (
    <CardPokemon key={p.id} data={p} />
  ))}
</AnimatePresence>
```

ONDE ESTÁ O FRAMER MOTION NO SITE

Em todas as páginas



Barra de progresso

A barra amarela do topo enche conforme você rola a página.

`useScroll + useSpring` · BarraProgresso.tsx



Transição entre páginas

A página antiga sobe e some; a nova entra de baixo para cima.

`AnimatePresence + exit` · TransicaoPagina.tsx



Logo da Pokédex

A Pokébola do menu gira 360° quando o mouse passa por cima.

`whileHover` · Titulo.tsx

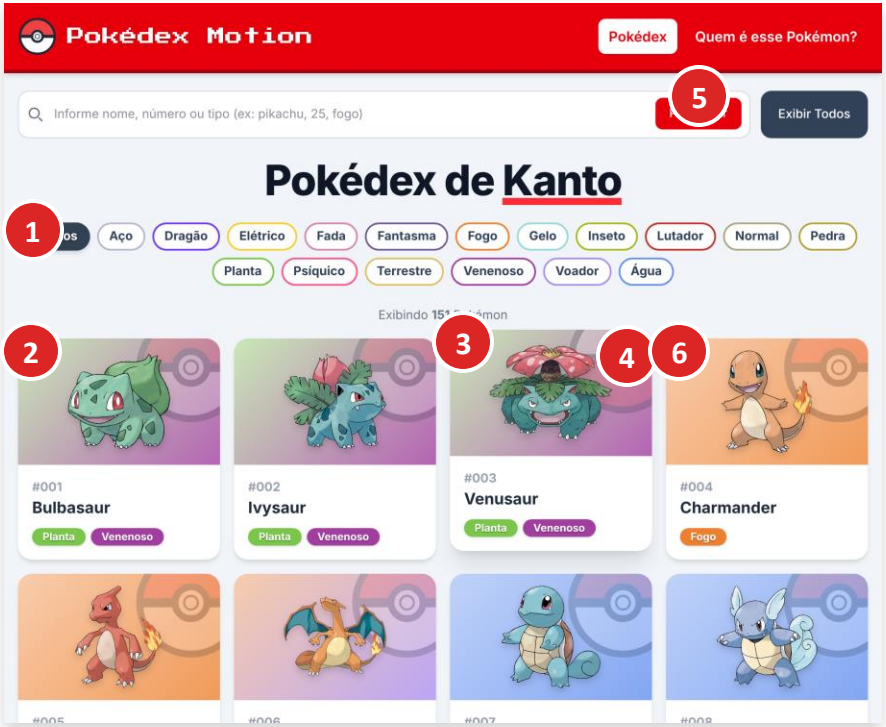


Carregando...

Enquanto os dados chegam, a Pokébola balança como numa captura.

`keyframes + repeat` · Pokebola.tsx

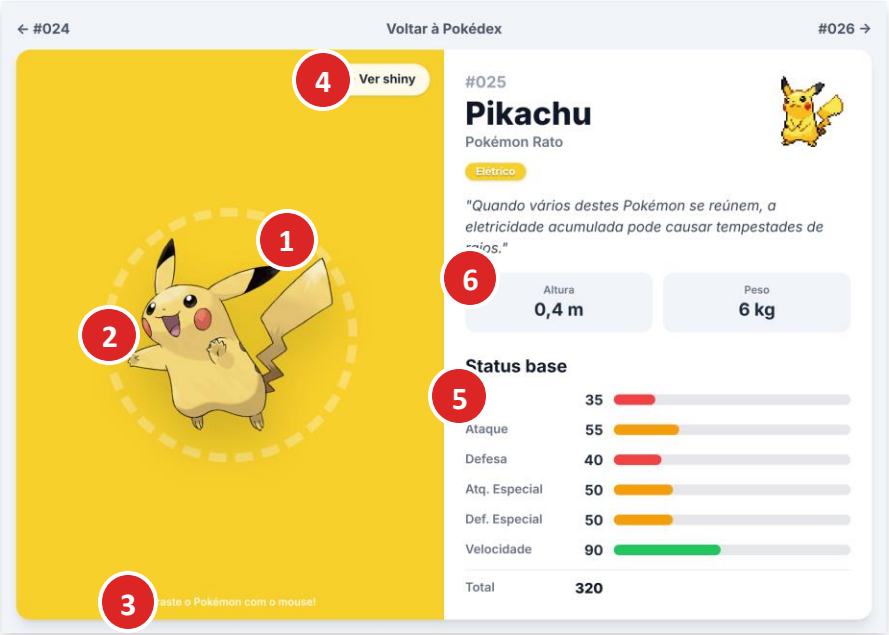
Página inicial (Pokédex)



Rota / · App.tsx

1. **Fundo do filtro desliza entre tipos**
   `layoutId` · FiltroTipos.tsx
2. **Cards surgem ao rolar a página**
   `whileInView` · CardPokemon.tsx
3. **Hover: o card sobe e a imagem balança**
   `variants + whileHover` · CardPokemon.tsx
4. **A Pokébola do fundo gira junto**
   `variants (propagação)` · CardPokemon.tsx
5. **Clique: card e botões afundam**
   `whileTap` · CardPokemon / InputPesquisa
6. **Ao filtrar, os cards saem e se reorganizam**
   `AnimatePresence + layout` · App.tsx

Página de detalhes



Rota /detalhes/:pokemonId · Detalhes.tsx

1. **O Pokémon flutua sem parar**
   `repeat: Infinity` · ImagemPokemon.tsx
2. **Círculo pontilhado gira ao fundo**
   `rotate: 360 em loop` · ImagemPokemon.tsx
3. **Arraste: ao soltar, ele volta**
   `drag + dragSnapToOrigin` · ImagemPokemon.tsx
4. **"Ver shiny" vira a imagem como carta**
   `AnimatePresence` · ImagemPokemon.tsx
5. **Barras de status crescem em cascata**
   `staggerChildren + spring` · BarraStatus.tsx
6. **Altura e peso crescem no hover**
   `whileHover` · Detalhes.tsx

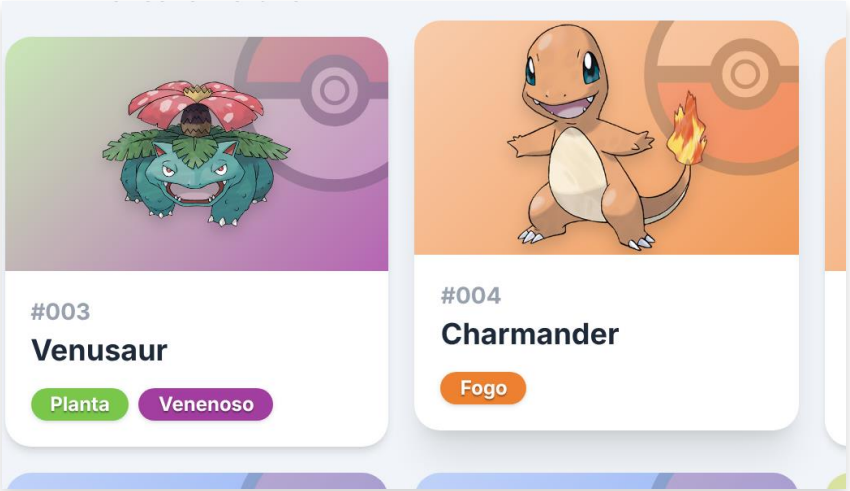
Quem é esse Pokémon?



Rota /quiz · Quiz.tsx

1. **Raios azuis giram no fundo**
   `rotate: 360 em loop` · Quiz.tsx
2. **O Pokémon entra girando**
   `initial/animate + spring` · Quiz.tsx
3. **As respostas entram em cascata**
   `staggerChildren` · Quiz.tsx
4. **Ao responder, a silhueta ganha cor**
   `filter: brightness` · Quiz.tsx
5. **Errou: treme · Acertou: pula**
   `useAnimate` · Quiz.tsx
6. **Botão "Próximo" surge crescendo**
   `AnimatePresence` · Quiz.tsx

CardPokemon: variants

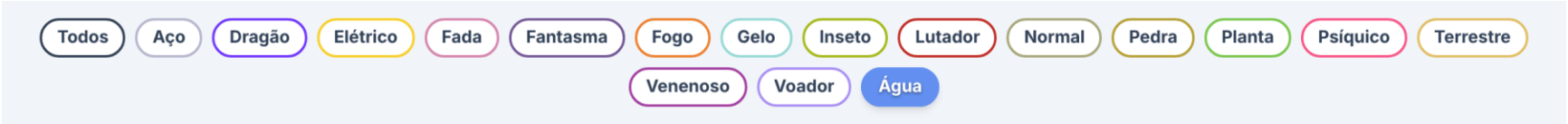


Mouse em cima: o card sobe, a imagem cresce e balança e a Pokébola do fundo gira — tudo com um único whileHover.

```
const variantesCard = {
  oculto:  { opacity: 0, y: 40 },
  visivel: { opacity: 1, y: 0 },
  saindo:  { opacity: 0 },
  hover:   { y: -10 },
}

<motion.div
  variants={variantesCard}
  initial="oculto"
  whileInView="visivel"
  whileHover="hover"
  whileTap={{ scale: 0.95 }}
  exit="saindo"
  layout
>
```

FiltroTipos: layoutId




```
{ativo && (
  <motion.span
    layoutId="tipo-ativo"
    className="absolute inset-0"
    style={{ backgroundColor: tipo.cor }}
  />
)}
```

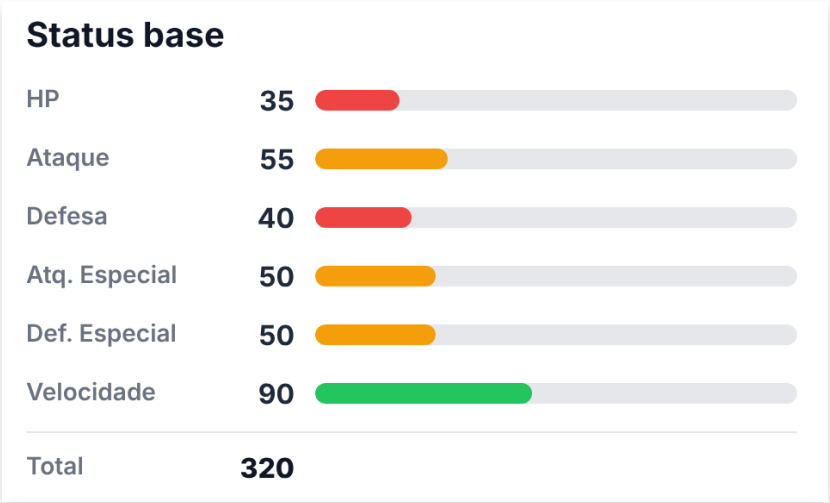
Só o botão ativo desenha o fundo colorido. Como todos usam o mesmo layoutId, o Framer Motion entende que é o mesmo elemento e faz o fundo deslizar até o botão clicado.

E os cards?

`<AnimatePresence mode="popLayout">`

Ao filtrar, os cards que saem rodam o exit; os que ficam deslizam para a nova posição por causa da prop layout.

BarraStatus: stagger



As barras crescem uma depois da outra (cascata) e "passam do ponto" antes de parar, como uma mola.

```
// pai: atrasa cada filho 0,12s
const variantesLista = {
  visivel: { transition: {
    staggerChildren: 0.12
  } }
}

// filho: largura vem do "custom"
const variantesBarra = {
  oculto: { width: 0 },
  visivel: (valor) => ({
    width: `${valor / 2}%`,
    transition: { type: "spring" }
  })
}
<motion.div custom={valor}
  variants={variantesBarra} />
```

Pokébola e drag



Keyframes + repeat
Loading: a Pokébola balança como na captura

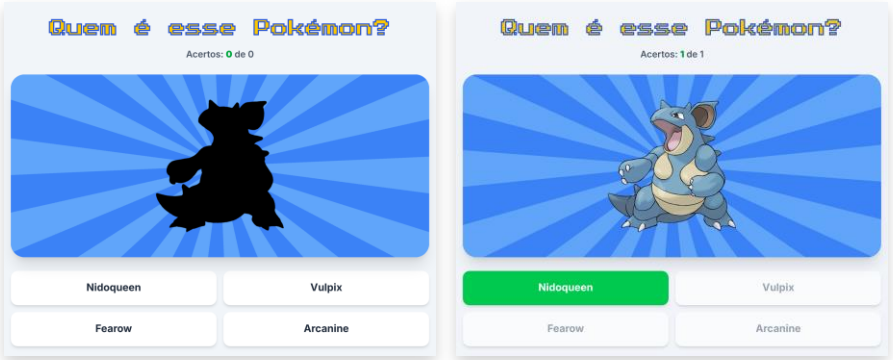
```
<motion.svg
  animate={{
    rotate: [0, -25, 25, -15, 15, 0]
  }}
  transition={{
    repeat: Infinity,
    repeatDelay: 0.5
  }}
/>
```



Drag
Arraste o Pokémon: ao soltar ele volta ao lugar

```
<motion.img
  drag
  dragConstraints={areaRef}
  dragSnapToOrigin
  whileDrag={{ scale: 1.15 }}
/>
```

Quiz: useAnimate



Silhueta: filter: brightness(0). Ao responder, o filtro anima até brightness(1) e o Pokémon aparece.

```
const [scope, animate] =
useAnimate()

function responde(opcao) {
  if (opcao == pokemon.nome) {
    // acertou: pula
    animate(scope.current,
      { y: [0, -30, 0] })
  } else {
    // errou: treme
    animate(scope.current,
      { x: [0, -25, 25, 0] })
  }
}
<div ref={scope}>...</div>
```

Transição e progresso

TransicaoPagina.tsx

```
const location = useLocation()

<AnimatePresence mode="wait">
  <motion.main
    key={location.pathname}
    initial={{ opacity: 0, y: 30 }}
    animate={{ opacity: 1, y: 0 }}
    exit={{ opacity: 0, y: -30 }}
  >
    <OutletCongelado />
  </motion.main>
</AnimatePresence>
```

A key muda a cada troca de rota: a página antiga roda o exit e a nova roda o animate.

BarraProgresso.tsx

```
const { scrollY, scrollYProgress } =
  useScroll()

// suaviza com efeito de mola
const scaleX = useSpring(scrollYProgress)

// 0 a 80px rolados => opacidade 0 a 1
const opacity = useTransform(
  scrollY, [0, 80], [0, 1])

<motion.div style={{ scaleX, opacity }} />
```

Motion Values: a barra acompanha a rolagem sem o React renderizar o componente de novo.

RESUMO

Onde está cada animação

Arquivo	Onde aparece	Recursos do Framer Motion
`CardPokemon.tsx`	Cards da Pokédex	variants, whileHover, whileTap, whileInView, layout, exit
`FiltroTipos.tsx`	Botões de tipo	layoutId (shared layout)
`App.tsx`	Grid de cards	AnimatePresence mode="popLayout"
`BarraStatus.tsx`	Página de detalhes	staggerChildren, custom, spring
`Pokebola.tsx`	Carregando / logo	keyframes, repeat: Infinity
`ImagemPokemon.tsx`	Página de detalhes	drag, dragSnapToOrigin, AnimatePresence
`Quiz.tsx`	Quem é esse Pokémon?	useAnimate, animação de filter
`TransicaoPagina.tsx`	Troca de páginas	AnimatePresence + key da rota
`BarraProgresso.tsx`	Topo da página	useScroll, useSpring, useTransform

Como rodar o projeto



Back-end (porta 3000)

```
cd framer-motion/back
npm install
// copiar .env.example p/ .env
npx prisma migrate dev
npx prisma db seed
npm run dev
```



Front-end (porta 5173)

```
cd framer-motion/front
// copiar .env.example p/ .env
npm install
npm run dev

// abrir: http://localhost:5173
```

Pré-requisitos: Node.js e PostgreSQL. Ajuste usuário e senha do banco no arquivo .env.

RESUMO

Roteiro da demonstração

1

Pokédex

Rolar a página: os cards surgem e a barra amarela enche.

2

Filtro

Clicar em Fogo, Água e Planta: o fundo desliza e os cards se reorganizam.

3

Cards

Passar o mouse e clicar: o card sobe, balança e afunda.

4

Detalhes

Ver as barras, arrastar o Pokémon, clicar em Ver shiny e ir ao próximo.

5

Quiz

Errar uma vez (a imagem treme) e acertar outra (ela pula).

Dica: deixe o PostgreSQL, o back-end (npm run dev) e o front-end (npm run dev) rodando antes de começar a apresentação.

RESUMO

Boas práticas

Com moderação

Animação deve guiar o usuário, não distrair. Durações curtas (0,2 a 0,5s).

Performance

Prefira animar transform (x, y, scale, rotate) e opacity: são os mais leves para o navegador.

Acessibilidade

<MotionConfig reducedMotion="user"> respeita quem desativa animações no sistema.

Motion Values (useScroll, useSpring) atualizam a tela sem re-renderizar o componente.



Obrigado!

Agora vamos para a demonstração da Pokédex

Código e apresentação no GitHub:

github.com/mateus-zanatta/TrabalhoFullStack-MortionFrames

